Day Final - Final Presentation

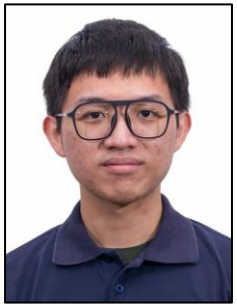
2026 / 11 / 04

OpenACC

More Science, Less Programming

Team Name: Parallel Robotics

Team Members



YuChung-Chen, NTU



YungShun-Chan, NTU

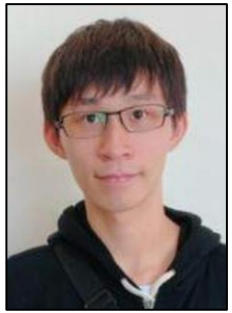


YuHsiang-Chen, NTHU



MinChih-Shih, NTHU

Team Mentors



Johnson-Sun, NVIDIA



Peggy-Chen, NVIDIA

OpenACC More Science, Less Programming

OPEN HACKATHONS

NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing

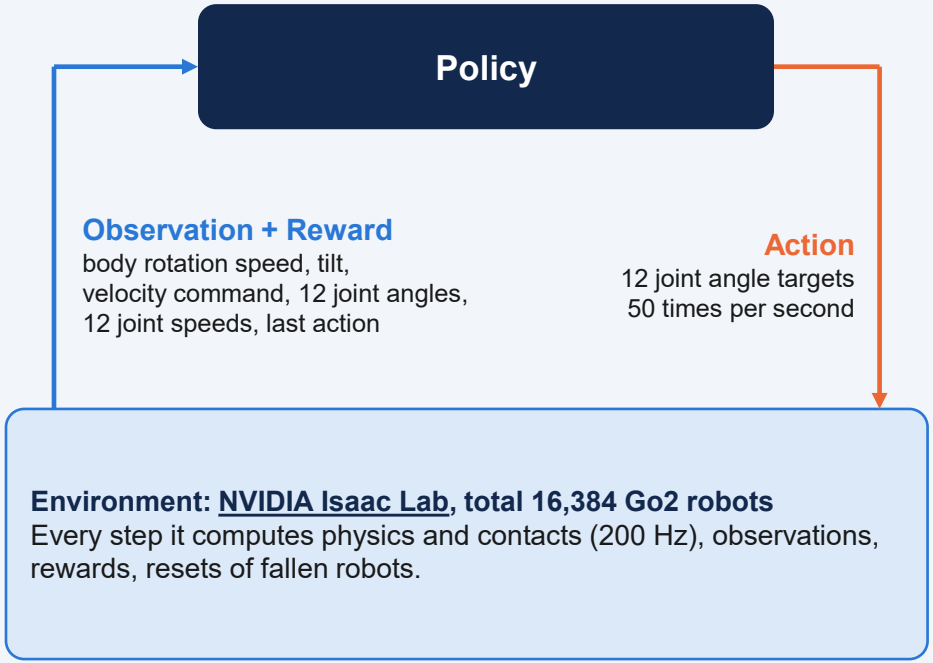
NVIDIA

What our Task Looks Like



Scalable RL Training for Quadruped Locomotion

A robot dog learns to walk by trial and error, in thousands of simulated copies at once



Main method: DreamWaQ (ICRA 2023).

Environment setup

Terrain: flat, rough, slopes, boxes, stairs.
Commands: forward, sideways and turn speed.
Randomization: friction, payload, motor delay.

Curriculum

Start easy. Robots that walk well move to harder terrain, robots that fall move back. The command range grows from ±0.5 to ±1.0.

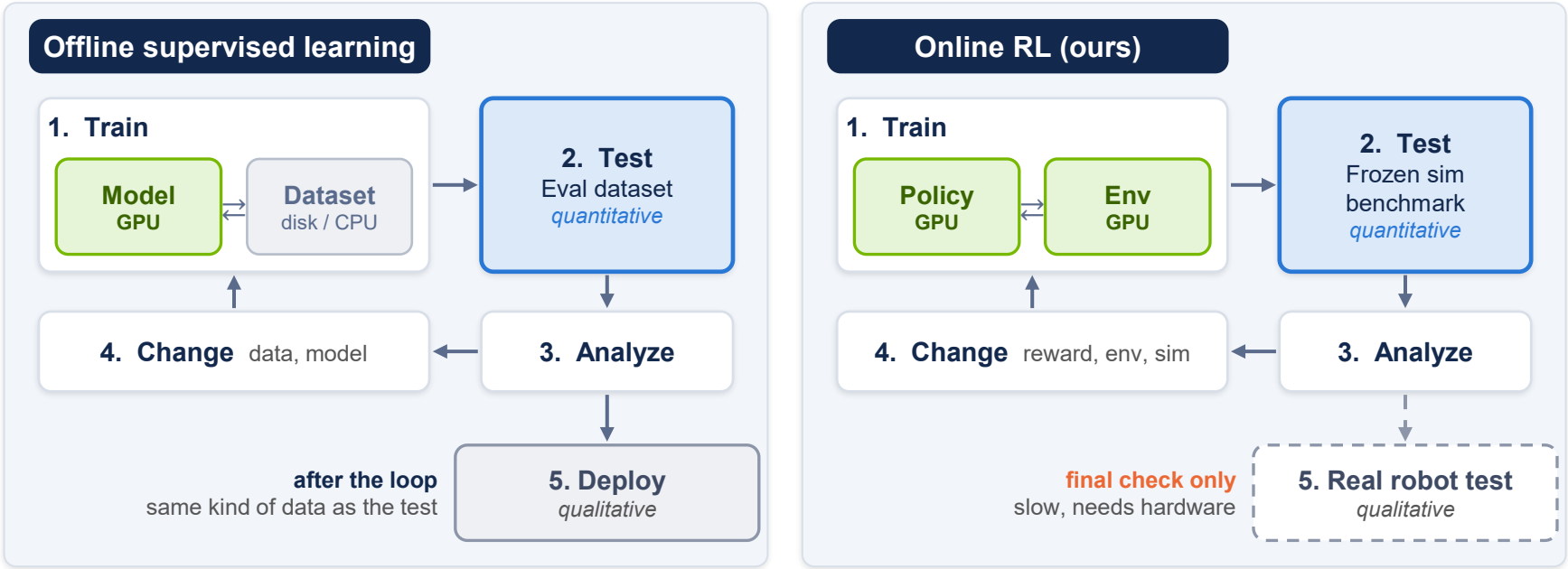
Reward

\+ follow the commanded speed and turn.
− tilting, shaking, jerky motion, slipping, high torque.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Offline Supervised Learning vs. Online RL

Same loop. The difference is the test and **what runs on the GPU**.



Same loop, more GPU work: in RL the environment runs on the GPU too, and here it is the slow part (physics 58% of an iteration, policy inference under 1%).

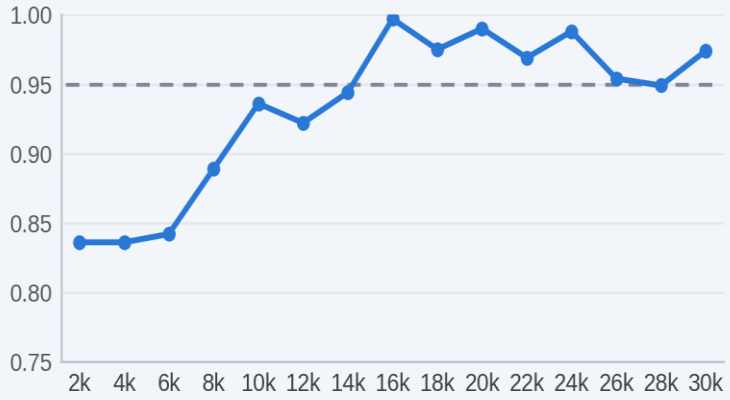
A test is only useful if it predicts real use: eval dataset or sim benchmark scores must track deployment performance.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Sim Evaluation: How We Measure Speed and Quality

The same frozen benchmark, run in PhysX and in Newton, scores every policy across weeks and branches

Rule: tracking must stay above the line



Example seed: first above 0.95 at 16k, but it dips at 28k, so it counts as staying ≥ 0.95 only from 30k.
Stage value = mean over 3 seeds.

Quality: frozen benchmark

- 384 robots, fixed terrain with slopes, obstacles and stairs
- 16 commands × 4 terrain groups, 60 s rollouts
- Same disturbances as training: friction, mass, pushes, noise

Tracking ratio

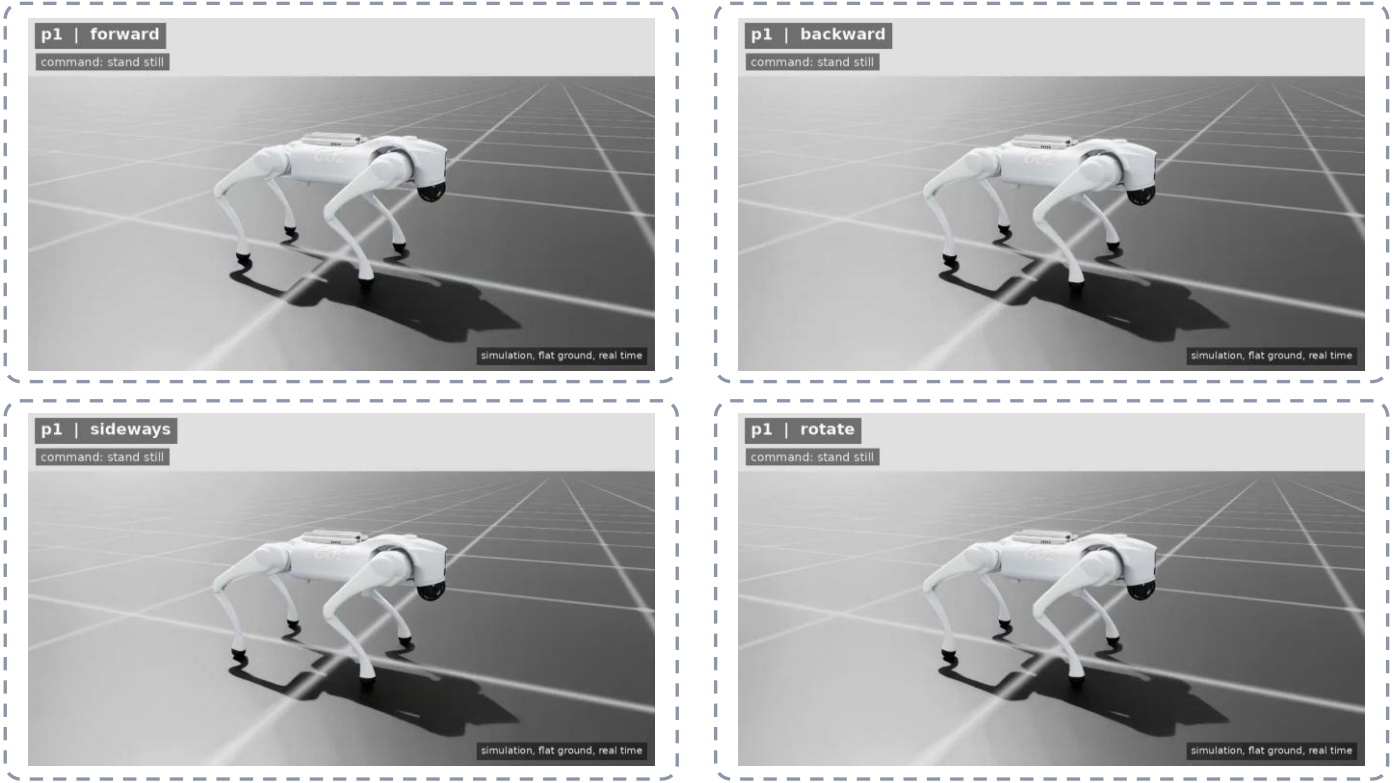
distance moved ÷ distance commanded

1.0 means the robot moved as far as commanded. Falls and stalls score low. A checkpoint's score is the mean over 52 scenario × terrain cells.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

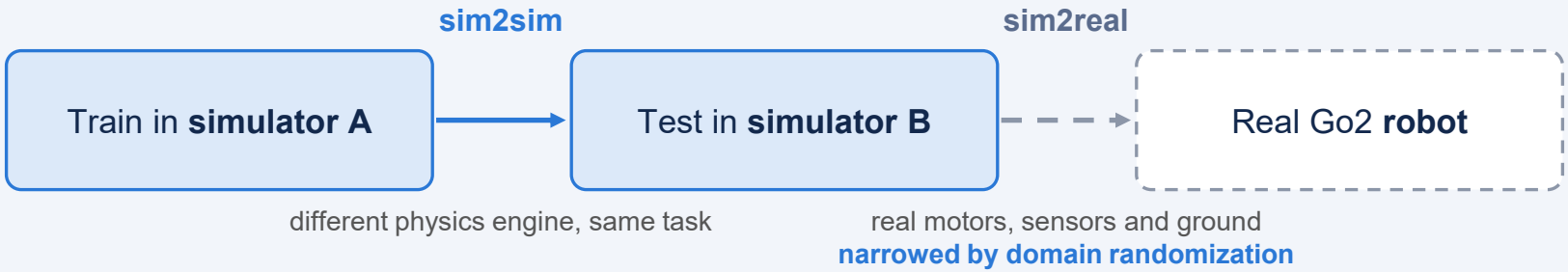
What the Benchmark Looks Like

Same frozen terrain and commands for every policy: 4 of the 16 command scenarios



Sim2Real Gap & Sim2Sim (across physics engines)

Testing across simulators shows whether a policy learned the task or one simulator's quirks



Our setup

Trained in ↓ Scored on →	PhysX benchmark	Newton benchmark
Trained in PhysX	same simulator	**sim2sim**
Trained in Newton	**sim2sim**	same simulator

Every policy runs on both benchmarks. Headline numbers in this talk use the PhysX benchmark.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

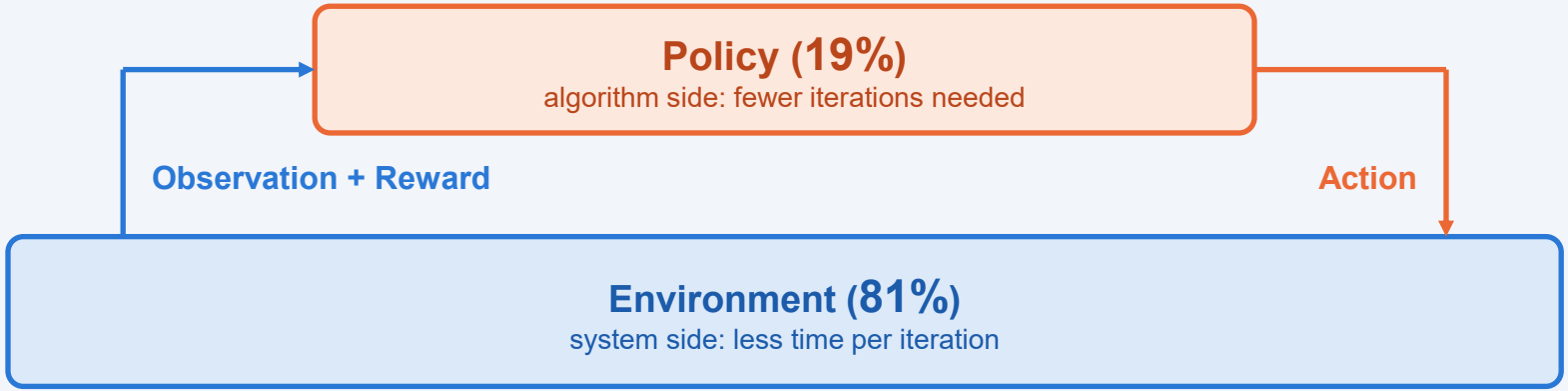
Evolution and Strategy

From a faster simulator to work on both the system and the learning algorithm

Wall time = iterations needed × time per iteration

Algorithm side (policy): more GPUs per update, symmetry augmentation, assist force, KL annealing

System side (environment): Isaac Lab 3.0, Newton MJWarp solver, code optimizations, Warp env with CUDA graphs

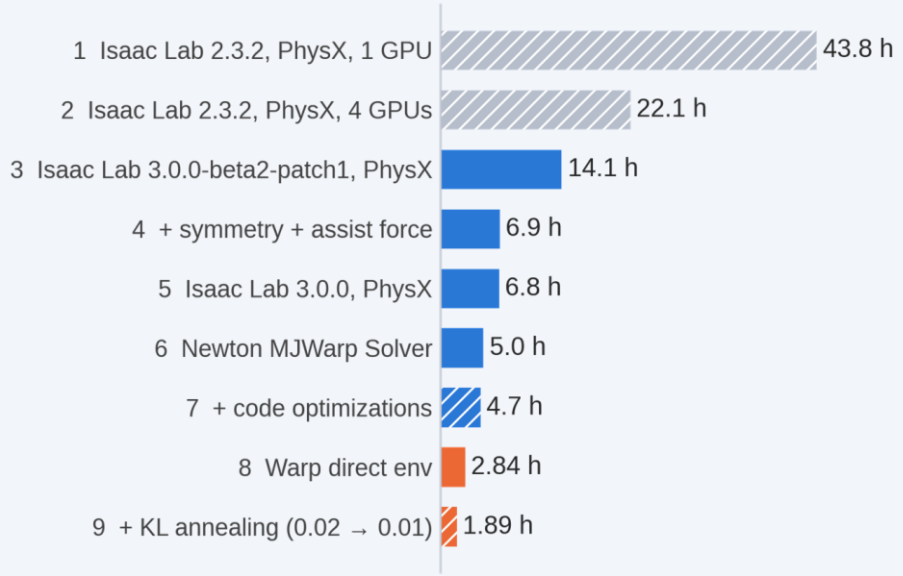


Shares of GPU busy time in the final setup (profile in backup). Main method: DreamWaQ (ICRA 2023).

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Results: A Good Policy in 1.89 h Instead of 43.8 h

Wall time until benchmark tracking stays ≥ 0.90, 4,096 envs/GPU, mean of 3 seeds (KL: 2 seeds)



Hatched: estimated. Steps 1–2 use a teammate's speed ratio, step 7 applies the measured 1.08x code speed-up, step 9 uses preliminary KL runs (2 seeds, another machine).

23.1x to a good policy (stays ≥ 0.90) 43.8 h → 1.89 h

14.7x to a very good policy (stays ≥ 0.95) 62.6 h → 4.26 h

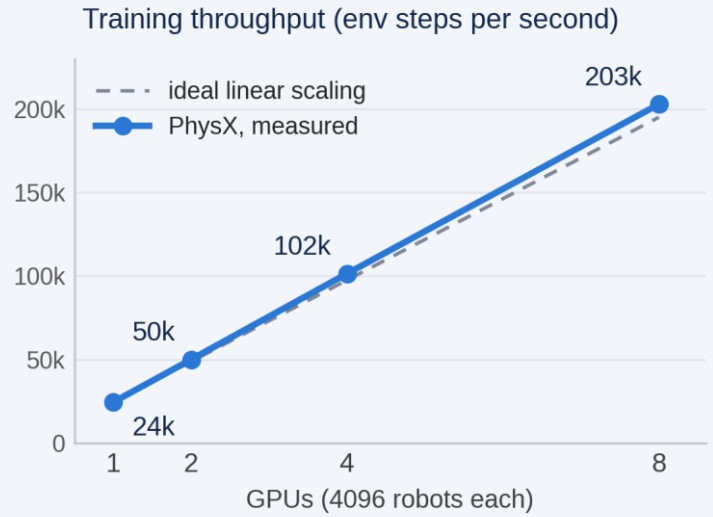
9.5 h for a full training run, was 2.6 days

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Steps 1–3: More GPUs, Then Isaac Lab 3.0.0-beta2

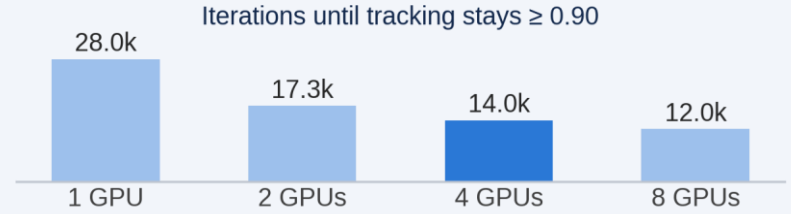
Wall time: 43.8 h → 22.1 h → 14.1 h

Throughput scales with GPUs



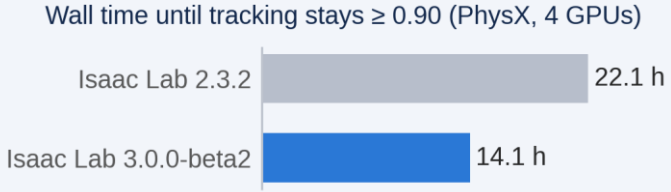
8 GPUs give 8.3x the throughput of one (104% efficiency), so an iteration takes about the same time but carries 4x the data on 4 GPUs.

More GPUs: fewer iterations



4x the data per iteration halves the iterations (28k → 14k).

Isaac Lab 3.0.0-beta2: faster iterations

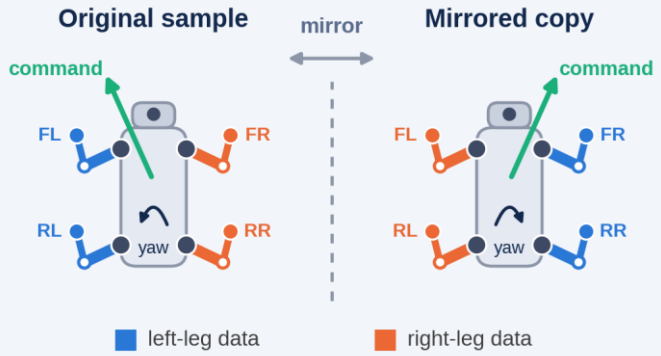


Same 14k iterations, each 1.56x faster (5.67 s → 3.63 s).

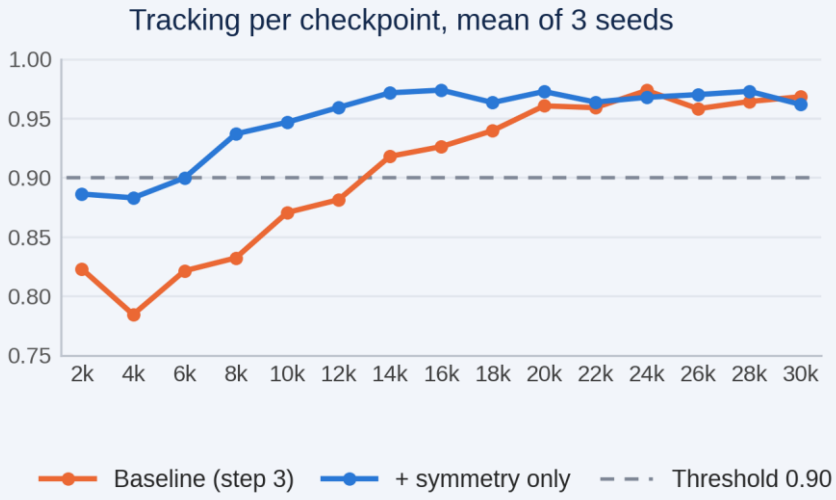
Step 4 (1/2): Symmetry Augmentation

The biggest single gain: 2.6x fewer iterations to a good policy

How it works



- The robot and the task are left-right symmetric, so a mirrored sample is also a valid sample.
- Each PPO minibatch is doubled with its mirror: swap left and right legs, flip sideways values (vy, yaw rate, gravity y).
- Twice the data per iteration with no extra simulation and no extra loss.



2.6x
fewer iterations
14k → 5.3k

5.9 h
to ≥ 0.90
was 14.1 h

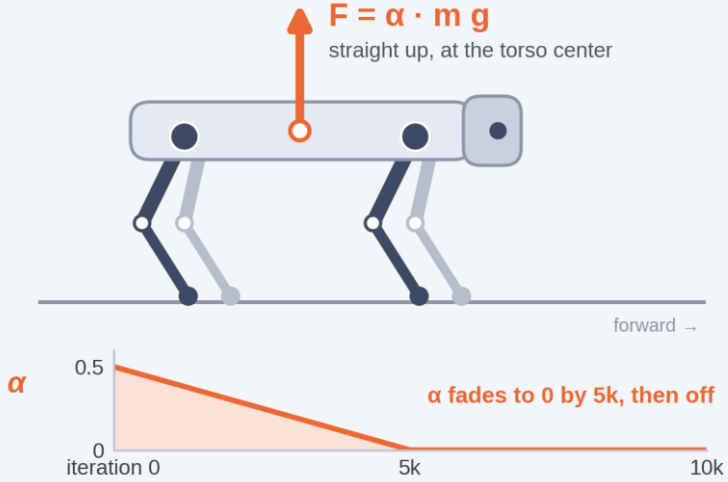
+9%
time per
iteration

OpenACC
More Science, Less Programming
OPEN HACKATHONS
NCHC
國家實驗研究院
國家高速網路與計算中心
National Center for High-performance Computing
NVIDIA

Step 4 (2/2): Assist Force in Early Training

Helps a young policy stay upright, but adds little on top of symmetry

What it does



Carries part of the weight early, so robots fall less and the terrain and command curricula advance sooner. Training only.

Idea from Watch Your Step (Heyrman et al. 2026)

Hours until tracking stays ≥ 0.95

	No assist	Assist force
No symmetry	26.2 h	13.7 h
Symmetry	12.4 h	9.2 h

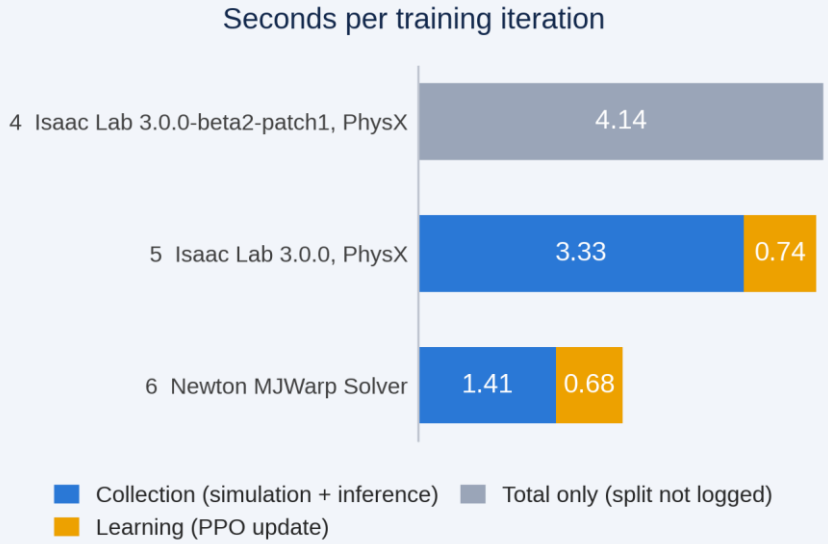
Alone: about 2x faster to 0.95 (26.2 → 13.7 h).

On top of symmetry: 0.95 sooner (11.3k → 8k iterations), 0.90 not (5.3k → 6k), +5% time per iteration.

Step 4 total: 14.1 → 6.9 h to ≥ 0.90 (2.05x).

Steps 5–6: GPU Physics with Newton MJWarp

Each training iteration went from 4.14 s to 2.09 s



Newton cuts the simulation time 2.4x (3.33 → 1.41 s). The PPO update stays at about 0.7 s.

Isaac Lab 3.0.0 (step 5)

The release that runs the Newton backend. PhysX speed is about the same: 4.14 → 4.06 s per iteration.

Newton MJWarp Solver (step 6)

GPU physics on NVIDIA Warp. All robots step in batched kernels captured in a CUDA graph.

2.4x **faster simulation**
collection 3.33 → 1.41 s

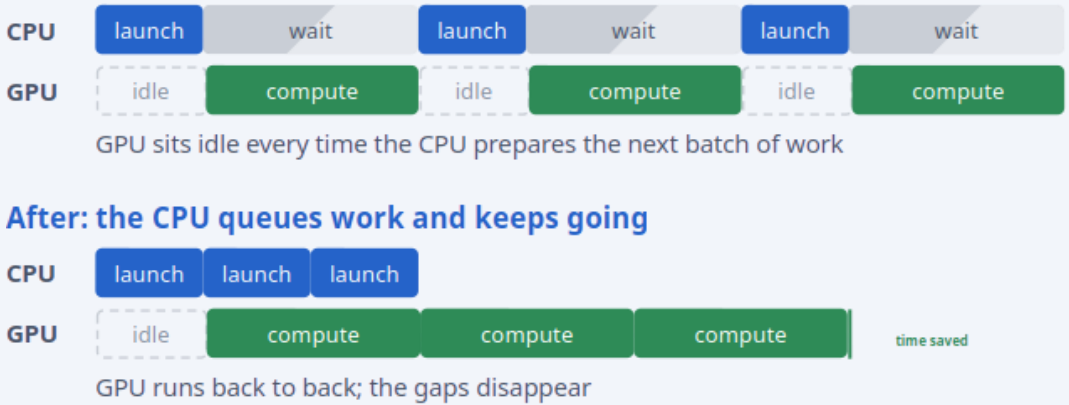
1.95x **faster per iteration**
4.06 → 2.09 s

1.35x **faster to a good policy**
6.8 → 5.0 h (needs 8.7k vs 6k iterations)

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Step 7: Code Optimizations

Keep the GPU busy: stop making the CPU wait



Keep results on the GPU

Logging and resets no longer read numbers back to Python every step

Don't re-read what never changes

Foot friction and contact data are fetched once, then reused

Keep bookkeeping on the GPU

Index lists live on the GPU, so nothing is copied over each step

1.10x

PhysX speed-up (4.06 to 3.68 s/iter), with bit-identical training results

1.08x

Newton speed-up on a 30K-iteration run (20.8 h to 19.3 h)

Next: Warp

The Warp version takes this to the limit: the whole env step runs on the GPU

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Step 8: Rewrite as Warp Direct Env

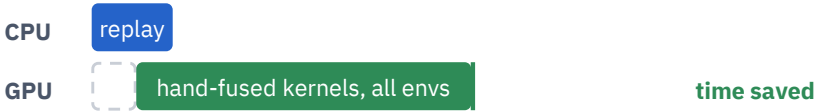
Run the whole env step on the GPU

PyTorch env: Python launches hundreds of small ops



Each op is a separate trip from Python; the GPU gets work in small pieces

Warp env: one launch replays the whole step



The GPU receives the step in one piece and runs it back to back

Three ideas

1 Hand-fused GPU kernels

All 18 reward terms in one Warp kernel, one thread per robot; 25 kernels replace hundreds of small ops

2 Record once, replay every step

A CUDA graph records the kernel sequence once; each step replays it with a single call

3 Share memory with physics

Kernels read Newton's state in place, so no data is copied and the CPU never waits

~1.2×

vs the PyTorch Newton env (about 1.91 to 1.58 s/iter on one H200)

86%

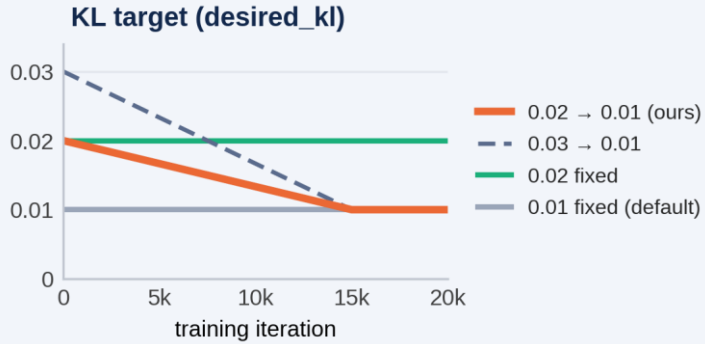
GPU busy while collecting data; env logic is only 45 ms of a 1.30 s rollout

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

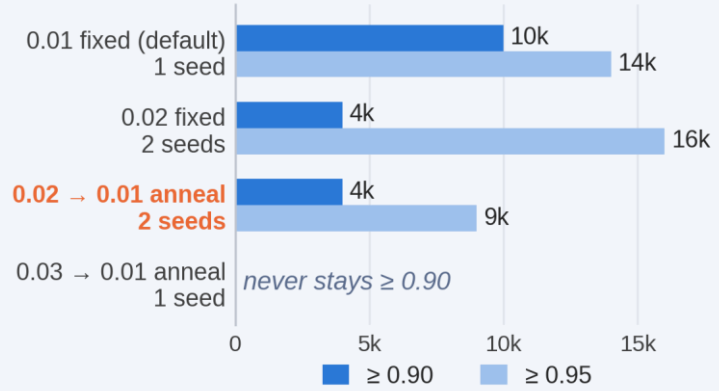
Step 9: KL Annealing

Big policy updates early, careful ones late: tracking ≥ 0.90 in 4k iterations instead of 10k

How it works



- PPO tunes its learning rate so each update changes the policy by about the KL target.
- Higher target: bigger steps, faster early learning, but noisy later.
- Annealing 0.02 → 0.01 by 15k gets both. Too high (0.03) never stabilizes.



2.5x
fewer iterations
to ≥ 0.90 (10k → 4k)

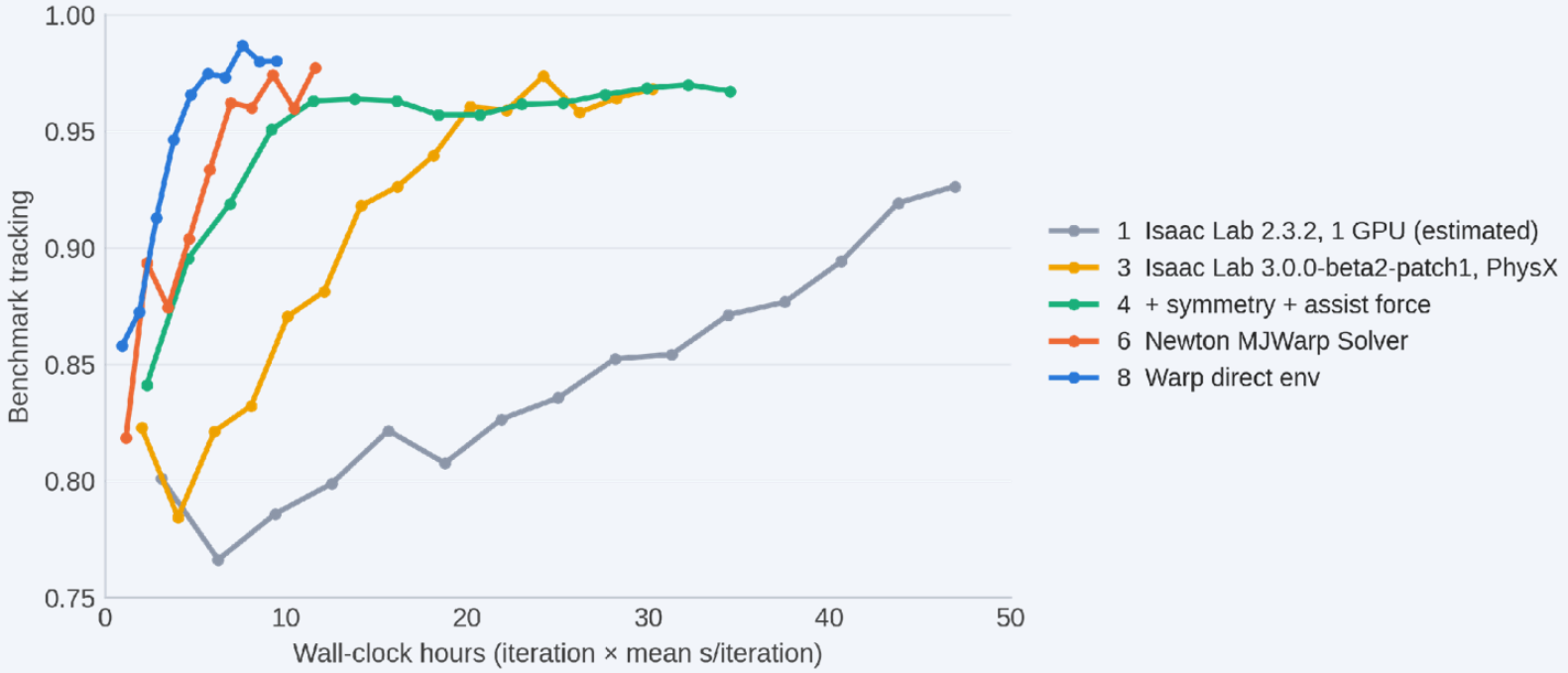
1.6x
fewer iterations
to ≥ 0.95 (14k → 9k)

1.89 h
L40 estimate to ≥ 0.90
was 2.84 h

Preliminary: 1–2 seeds per setting on one machine. Only the KL target changes, so the time per iteration is the same.

Same Quality, Much Sooner

Benchmark tracking vs wall-clock training hours, mean of 3 seeds. The curves move left and end higher.



Step 9 (KL annealing) is not drawn: its runs used another machine.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Where the 23x Comes From

Each step against the one before, wall time until tracking stays ≥ 0.90

Step	What changed	Wall time	Factor	Mainly through
1 → 2	1 GPU → 4 GPUs	43.8 → 22.1 h	1.99x	fewer iterations (4x data per iteration)
2 → 3	Isaac Lab 2.3.2 → 3.0.0-beta2-patch1	22.1 → 14.1 h	1.56x	faster iterations
3 → 4	+ symmetry augmentation + assist force	14.1 → 6.9 h	2.05x	fewer iterations (2x data per iteration)
4 → 5	Isaac Lab 3.0.0 release	6.9 → 6.8 h	1.02x	same speed, iterations assumed equal
5 → 6	PhysX → Newton MJWarp Solver	6.8 → 5.0 h	1.35x	1.95x faster iterations, more iterations
6 → 7	+ code optimizations	5.0 → 4.66 h	1.08x	faster iterations (measured on Newton)
7 → 8	Warp direct env	4.66 → 2.84 h	1.64x	faster and fewer iterations
8 → 9	+ KL annealing (0.02 → 0.01)	2.84 → 1.89 h	1.50x	fewer iterations (preliminary, 2 seeds)

1.99 × 2.05 × 3.8 × 1.50 ≈ 23x

1.99x
More GPUs
4x the data per iteration

2.05x
Symmetry + assist force
mirrored data, support early on

3.8x
GPU-native stack
Isaac Lab 3.0, Newton, code opt., Warp env

1.50x
KL annealing
bigger updates early (preliminary)

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Energy Efficiency

Estimated with the official calculator's method, GPU power only

5.8x

less GPU energy per trained policy

13.1 → 2.3 kWh

speedup 23.1x × 1 GPU ÷ 4 GPUs

INPUTS	
Baseline	1 × L40, Isaac Lab 2.3.2, PhysX
Final	4 × L40, Warp env + KL annealing
Application speedup	**23.1x**

GPU ENERGY (300 W per L40)			
	Baseline	**Final**	**Savings**
GPU-hours per policy	43.8	7.6	5.8x fewer
GPU energy per policy	13.1 kWh	2.3 kWh	10.9 kWh
GPU power, same throughput	6,930 W	1,200 W	5,730 W
Annual energy, same rate	60,700 kWh	10,500 kWh	50,200 kWh
Power efficiency			**5.8x**

Estimate: GPU board power only, excludes CPU, memory, network and cooling. Same method as the official calculator: speedup × baseline GPUs ÷ final GPUs.

Final Thoughts

What we take away and what comes next

1

Was it worth it?

Yes. A full DreamWaQ training run went from 2.6 days to 9.5 h, with equal or better quality.

2

Will we continue?

Yes. This stack is now our default for locomotion training, and other tasks (depth) will move to the Warp env.

3

Next steps

Test the new policies on the real robot, forecast 8-GPU or multi-node training, and optimize for the edge device.

4

Support we need

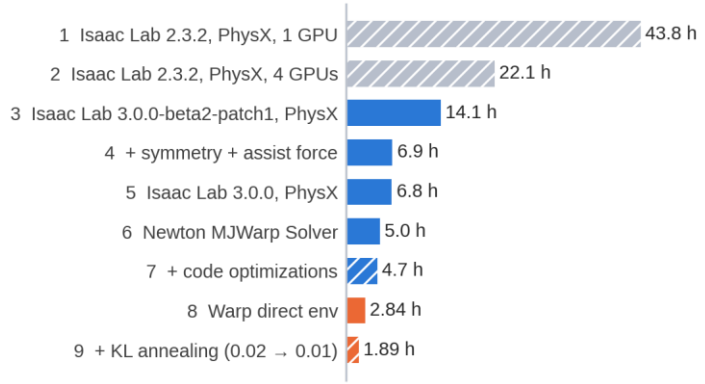
GPU cluster time for studies, and NVIDIA help on Newton and Warp.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC
國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Application Background

Application: a blind walking policy (DreamWaQ) for the quadruped robot, trained with reinforcement learning in NVIDIA Isaac Lab. It follows velocity commands on stairs, slopes and rough ground using only body sensors

Motifs: thousands of robots simulated in parallel on the GPU, PPO with an asymmetric actor-critic, data-parallel multi-GPU training



Hours until benchmark tracking stays ≥ 0.90, from Isaac Lab 2.3.2 on 1 GPU (top) to the Warp env with KL annealing on 4 GPUs (bottom).

Hackathon Objectives and Approach

Programming models: PyTorch, NVIDIA Warp kernels, CUDA graphs, data-parallel PPO (torchrun)

Profiling/hot spots: GPU busy only 18% in the torch env, ~186 syncs per step

Refactoring: env logic moved into Warp kernels, no CPU-GPU sync in a step

Libraries: Isaac Lab 3.0, Newton (MuJoCo-Warp), RSL-RL

Performance tuning: 1 → 4 GPUs, symmetry augmentation, assist force, KL annealing

Other: a frozen benchmark checks quality for every change

Technical Accomplishments and Impact

Speedup: 43.8 h → 1.89 h to a good policy (23.1x), 62.6 h → 4.26 h to a very good one (14.7x)

How: more GPUs, symmetry augmentation, KL annealing and a GPU-native stack (Isaac Lab 3.0, Newton, code optimizations, Warp env)

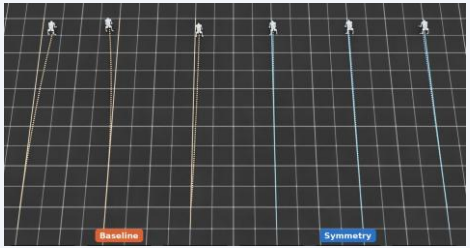
Quality: a full run went from 2.6 days to 9.5 h with equal or better tracking

Why it matters: a 3-seed A/B test now fits in a day, so we can try more rewards, simulators and robot setups

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC
國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

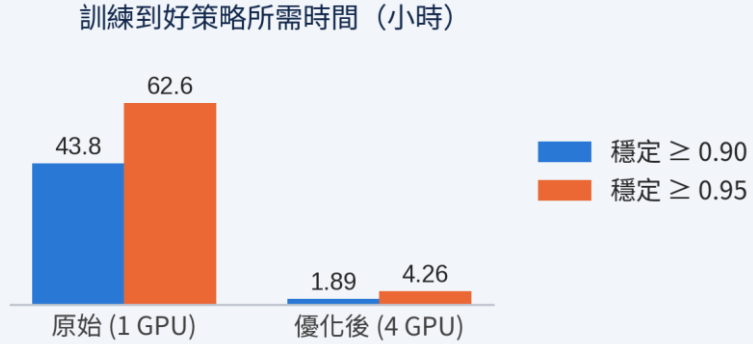
讓機器狗更快學會走路：四足機器人訓練加速 23 倍

Parallel Robotics 團隊來自李濬屹、楊元福老師帶領的 ELSA LAB，
AI Agent 將程式最佳化、訓練排程的工作流程加速了約 30 倍；GPU 將四足機器人行走策略的訓練加速了 23.1 倍！！



要求直走時，未用對稱增強的策略會偏向一側（左三隻），用了之後幾乎走直線（右三隻）。

四足機器狗要在階梯、斜坡和碎石上穩定行走，需要在模擬器裡讓數千隻機器狗同時練習，再用強化學習更新策略。原本在一張 GPU 上訓練一次要 62.6 小時，每個設計問題又要多個隨機種子比較，一次實驗就要好幾天。這次黑客松我們讓每一輪更快（多 GPU、Isaac Lab 3.0、Newton 物理引擎、程式最佳化、Warp 環境與 CUDA Graph），也讓需要的輪數更少（左右對稱資料增強、KL 調參）。訓練到好策略的時間從 43.8 小時縮短到 1.9 小時，品質不降反升，一天內就能完成過去要一週的比較。



23.1x
穩定 ≥ 0.90 加速

14.7x
穩定 ≥ 0.95 加速

5.8x
GPU-hours 減少

完整訓練 62.6 → 9.5 小時，部分數值為推估值，KL 為初步結果。

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA


Thank You
OpenACC
More Science, Less Programming